



ENTERPRISE REFERENCE GUIDE

AI Chatbot System

for the LMS Portal

A Technical Design, Implementation & Enterprise Scale-Up Playbook

RAG • Microservices • Multi-Modal AI • MongoDB • FastAPI • React

Project	AI Chatbot System for LMS Portal
Prepared By	Jatin Verma — Associate Intern, AI Chatbot Systems & Software Dev
Organisation	Reliance Jio Platforms Limited (Jio JPL)
Industry Mentors	Mr. Jeetendra Pradhan (GM) & Mr. Pradeep Tiwari (VP)
Faculty Mentor	Dr. Preeti Gupta, Head STME, NMIMS
Internship Period	January 2026 – July 2026 (16-week core build)
Document Type	Internal Engineering Reference Guide / Playbook
Classification	INTERNAL USE — Jio Platforms Limited

EXECUTIVE SUMMARY

Reliance Jio's enterprise Learning Management System (LMS) hosts thousands of Standard Operating Procedures (SOPs), training modules, and policy documents. As this repository grew, employees struggled to find precise information using traditional keyword search — a problem that translates directly into lost productivity, repeated queries to HR and managers, and inconsistent compliance with documented procedures.

The AI Chatbot System for the LMS Portal, built during a 16-week Associate Internship (January–July 2026), resolves this by giving employees a conversational interface — text, voice, or image — where every answer is grounded in the organisation's own SOP documents through Retrieval-Augmented Generation (RAG).

What Was Built

- A three-tier microservices architecture: React.js frontend, FastAPI Chatbot Service (Port 9000), and MCP V3 Server (Port 8100), communicating over REST/JSON.
- A RAG pipeline that routes each query to the most relevant SOPs for the employee's job role ('Hive Router'), retrieves text from MongoDB, and generates a grounded answer via Groq LLaMA 3.1 – 8B Instant.
- Multi-modal input: typed text, browser-based voice transcription (Web Speech API), and image understanding (Groq Vision — LLaMA 4 Scout).
- A document ingestion pipeline supporting 12 file formats (PDF, DOCX, XLSX, images, and more) with automated text extraction and sanitisation.
- Automated real-time response evaluation scoring every answer on Accuracy, Relevance, and Grounding — surfaced in a live Analytics Dashboard.
- A 9-collection MongoDB data layer covering user profiles, SOPs, skills, evaluations, and conversation history.
- 35+ REST API endpoints, an Admin Panel for user/SOP/skill management, and role-based access control.

Current Status & Evidence of Performance

The system reached 100% of its planned Week-16 milestones: both backend services operational, all four frontend modules complete, all nine database collections live, and all three input modalities functional. Against a pilot batch of 39 evaluated queries the system recorded an average Accuracy of 0.76, Relevance of 0.77, and Grounding of 0.67, with a composite AI System Health score of 73.5 / 100.

Architecture	Microservices — React + 2× FastAPI over MongoDB
Core AI	RAG with Groq LLaMA 3.1 (text) and LLaMA 4 Scout (vision)
Input Modes	Text, Voice (Web Speech API), Image (Base64 + Vision API)
API Surface	35+ REST endpoints across Port 8100 (MCP) and Port 9000 (Chatbot)
Data Layer	MongoDB, 9 collections, indexed for SOP and user-context lookups

Pilot Result	0.76 Accuracy / 0.77 Relevance / 0.67 Grounding over 39 queries
--------------	---

CHAPTER 1

Introduction & Problem Statement

1.1 Background

The rapid expansion of digital knowledge repositories within enterprises has created a significant challenge for employees seeking to access relevant information efficiently. At Reliance Jio Platforms Limited, the LMS hosts thousands of SOPs, training modules, organisational policies, and skill-development resources. As these repositories grow in size and complexity, employees increasingly struggle to navigate and retrieve the precise information they need — leading to reduced productivity and engagement.

This project addresses that challenge by designing and developing an AI-powered Chatbot System for the Jio LMS Portal. The system leverages NLP, LLMs, and Retrieval-Augmented Generation (RAG) to give employees an intuitive conversational interface for accessing organisational knowledge — responding in natural language to text, voice, and image inputs, with every answer grounded in verified SOP data.

1.2 Problem Statement

Traditional keyword-based search mechanisms are inadequate for handling contextual, role-specific, and conversational queries. The sheer volume of SOPs, policies, training materials, and organisational procedures within the LMS portal makes it increasingly difficult to retrieve the right information quickly, leading to time wastage, employee frustration, and reduced compliance with organisational knowledge standards.

Core Problem: Employees know the answer exists somewhere in the SOP library — they just can't find it fast enough, in the form they need, for their specific role.

1.3 Project Objectives

1. Develop an intelligent AI chatbot that enables employees to query the LMS portal using natural conversational language.
2. Implement RAG so that all chatbot responses are accurate and grounded in verified SOP documents.
3. Build a role-based response generation system that tailors information to the user's job role and organisational context.
4. Support multi-modal inputs — text, voice, and image queries — for enhanced user interaction.
5. Develop a scalable microservices-based backend architecture separating user management, document handling, and AI processing.
6. Create an intuitive React.js frontend with a chat interface, admin panel, and performance analytics dashboard.
7. Implement an automated real-time evaluation system to monitor chatbot response quality across accuracy, relevance, and grounding metrics.

8. Ensure the system is production-ready, modular, and extensible for future enterprise enhancements.

1.4 Internship Overview

The internship was undertaken at Reliance Jio Platforms Limited from January 2026 to July 2026. The author was placed in the AI Chatbot Systems and Software Development Team, working at the intersection of AI engineering, backend development, and enterprise system integration. The project evolved through three structured review cycles:

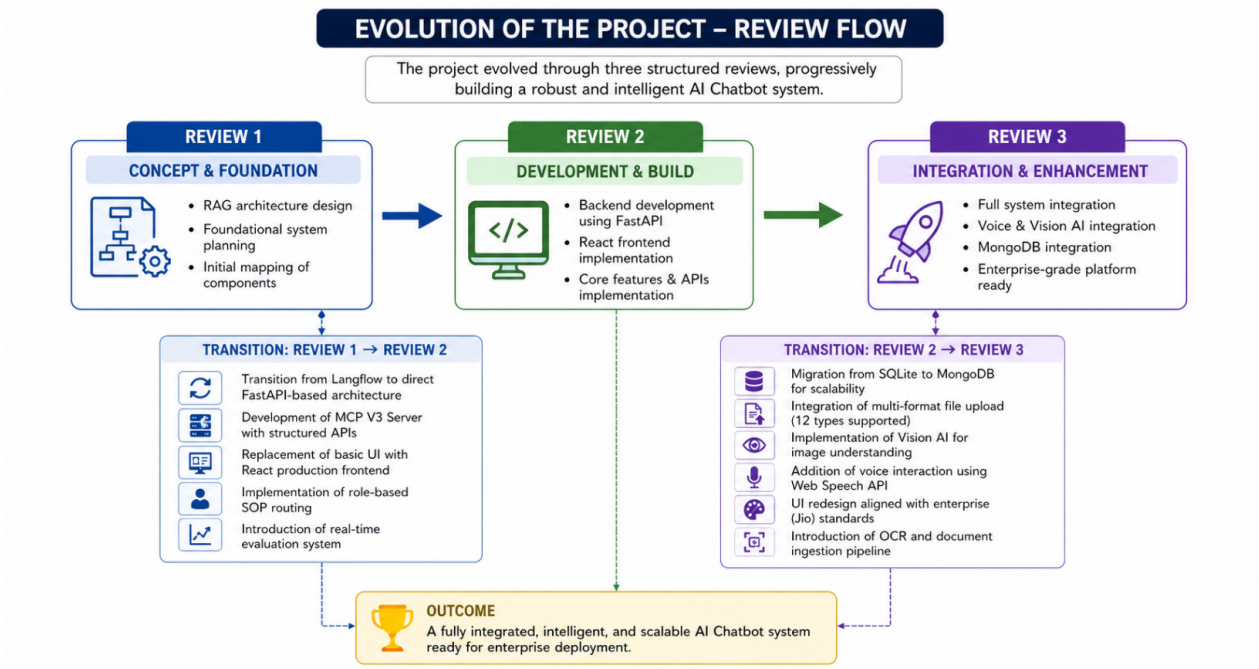


Figure 1.1: Evolution of the Project — Three-Phase Review Flow

Phase	Weeks	Focus	Key Outcome
Review 1	1–4	Concept & Foundation	RAG architecture; system planning; LangChain/Langflow exploration
Review 2	5–14	Development & Build	FastAPI microservices; MCP V3 Server; React frontend; role-based SOP routing; evaluation engine; 35+ APIs
Review 3	15–16	Integration & Enhancement	MongoDB migration; Voice & Vision AI; 12-format uploads; OCR; enterprise UI redesign

CHAPTER 2

Market Survey & Competitive Analysis

2.1 Existing Enterprise LMS Chatbot Solutions

A comprehensive market survey was conducted to understand the existing landscape of AI-powered chatbot solutions for enterprise Learning Management Systems, examining commercial products, open-source frameworks, and academic research to identify gaps and best practices.

Commercial Platforms

IBM Watson Assistant, Microsoft Azure Bot Service, and Salesforce Einstein Bots represent enterprise-grade solutions that integrate conversational AI into corporate knowledge platforms. However, these are primarily designed for general customer-service use cases and lack deep integration with role-based organisational knowledge structures. They often require extensive customisation and substantial licensing costs, making them impractical for internal SOP-driven enterprise deployments.

Open-Source Frameworks

Rasa, Botpress, and Haystack provide more flexibility for customisation. Rasa offers a dialogue-management framework for intent-classification chatbots but requires significant training data and lacks native LLM integration. Haystack provides strong RAG pipeline capabilities. LangChain and LlamaIndex represent newer frameworks designed specifically for LLM-orchestrated RAG pipelines — both were evaluated during Phase 1 before the strategic pivot to a direct FastAPI architecture.

2.2 Comparative Analysis

Feature	IBM Watson	Rasa	LangChain	Haystack	This System
Role-Based Routing	Partial	No	Partial	No	YES
LLM Integration	Yes	No	Yes	Yes	YES
Voice Input	No	No	No	No	YES
Vision / Image	No	No	No	No	YES
Custom Eval System	No	No	No	Partial	YES
Open Source	No	Yes	Yes	Yes	YES
Enterprise-Ready	Yes	Partial	Partial	Partial	YES
Multi-Format Docs	Partial	No	Partial	Yes	YES (12 formats)

Table 2.1: Comparison of Existing Enterprise LMS Chatbot Solutions

2.3 Gap Analysis & Justification

The market survey revealed a significant gap: no existing platform combines role-based SOP routing, multi-modal interaction, real-time quality evaluation, and native LLM integration in a single cohesive system. Existing platforms either require significant customisation effort, carry prohibitive licensing costs, or lack domain-specific role-based knowledge routing required for organisational SOP management.

Architecture Decision Rationale:

FastAPI was chosen over Django/Flask for its async-first design and auto-generated OpenAPI docs. Groq was chosen over direct OpenAI for ultra-low inference latency (critical for a real-time enterprise chatbot). MongoDB was chosen over PostgreSQL/SQLite for its flexible schema — SOP documents vary significantly in structure — and for horizontal scalability at enterprise scale.

CHAPTER 3

Technology Stack

The system uses a carefully selected modern stack, with each technology chosen for a specific engineering reason:

Layer	Technology	Version / Notes	Purpose
Frontend	React.js (Hooks, Components)	Latest stable	Chat UI, Admin Panel, Dashboard, Login
Styling	Tailwind CSS	Latest stable	Responsive, utility-first UI design
Data Viz	Recharts	Latest stable	Analytics dashboard charts (accuracy trend, verdict distribution)
Backend API	FastAPI (Python)	0.100+, Python 3.11+	High-performance async REST services (both microservices)
ASGI Server	Uvicorn	0.23+	Production-grade ASGI server for FastAPI
Database	MongoDB	7.x	Primary NoSQL data store — 9 collections
ODM	PyMongo	4.x	Python MongoDB driver for direct collection access
Text LLM	Groq API — LLaMA 3.1 – 8B Instant	Groq SDK latest	RAG text generation (Temperature 0.7, Max Tokens 2000)
Vision LLM	Groq API — LLaMA 4 Scout	Groq SDK latest	Image analysis and understanding
Voice	Web Speech API	Browser-native (W3C)	Speech-to-text; no external dependency or API key needed
PDF Processing	pdfplumber, PyPDF2	Latest stable	PDF text extraction with layout awareness
DOCX Processing	python-docx	1.x	Word document text extraction
XLSX Processing	openpyxl	3.x	Excel file data extraction
Image Files	Base64 + Groq Vision API	—	Image files analysed via LLaMA 4 Scout
OCR	Tesseract OCR (pytesseract)	5.x	Scanned PDF / image text extraction

Layer	Technology	Version / Notes	Purpose
CORS Middleware	FastAPI CORSMiddleware	Built-in	Cross-origin request handling (frontend ↔ backend)
Architecture Style	Microservices (MCP V3 + Chatbot)	—	Independent scalable services on separate ports
API Protocol	REST (JSON over HTTP/HTTPS)	—	Inter-service and frontend-to-backend communication
Frontend Hosting	npm serve (pilot)	—	Static build serving; production: Nginx / CDN

Table 3.1: Complete Technology Stack Summary

Enterprise Upgrade Note (Scale-Up):

For an enterprise-wide deployment, replace Groq Cloud with a self-hosted or VPC-hosted LLM (e.g., Ollama running LLaMA 3 on GPU nodes) to eliminate external API dependency and data-egress risk. Replace the current keyword-scoring SOP retrieval with semantic vector search (ChromaDB or Pinecone) for significantly improved retrieval accuracy at larger SOP volumes. See Chapter 13 for the full scale-up checklist.

CHAPTER 4

System Architecture

4.1 Overview

The final AI Chatbot System uses a three-tier microservices-based architecture, ensuring scalability, modularity, and ease of maintenance. The three primary tiers are: the React Frontend (presentation layer), the Backend Services Layer (two independent FastAPI microservices), and the Data Storage Layer (MongoDB + local file storage). Each component operates independently while communicating through well-defined RESTful APIs.

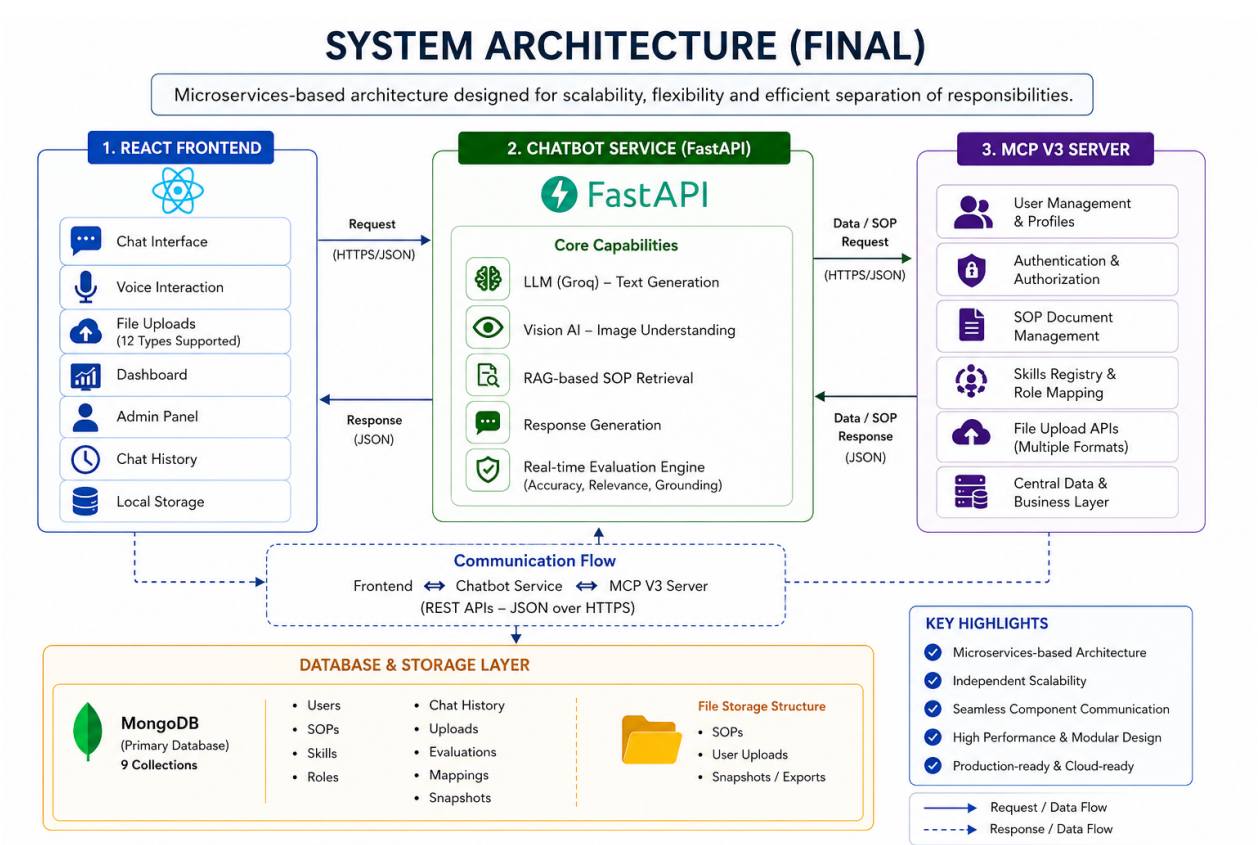


Figure 4.1: High-Level System Architecture (Final — Review 3)

4.2 Frontend Layer — React.js

The frontend is developed using React.js with a component-based, Hooks-driven architecture. It includes four primary modules:

- **Login Page:** Employee authentication by UID; role and profile data retrieved from MCP V3 Server on login.

- **Chat Interface:** Real-time text, voice, and image interaction; conversation history persisted via localStorage; SOP source citations shown per response; suggested follow-up questions generated by the LLM.
- **Admin Panel:** Complete control over user profiles, SOP document uploads, skills management, and role-to-skill mapping across five tabs (Users, SOPs, Skills, Files, Stats).
- **Analytics Dashboard:** Visual KPIs including accuracy, relevance, and grounding scores; verdict distribution bar chart; accuracy trend line chart; top queries; failure analysis; and PDF report generation.

4.3 Backend Services Layer

The backend consists of two independent FastAPI microservices. This separation is a deliberate architectural decision: it allows each service to be scaled, updated, and debugged independently, which is critical at enterprise scale.

Service	Port	Responsibility	Key Capabilities
MCP V3 Server	8100	Central data management layer	User profiles with RBAC; SOP registry (upload, versioning, metadata, retrieval); skills registry; role-to-skill mapping; user authentication
Chatbot Service	9000	AI processing and response generation	RAG pipeline execution; Groq LLM integration (text + vision); response evaluation engine; /chat and /chat/vision endpoints; performance dashboard data feed

4.4 Data Storage Layer

The data layer uses MongoDB (9 collections) as the primary database. In the final phase (Review 3), indexing and query optimisation were applied to improve retrieval speed for SOP lookups and user-context fetching — both critical on the critical path of every chatbot response. Local file storage handles SOP document files, user-uploaded content, and evaluation snapshots.

4.5 Deployment Topology & Port Map

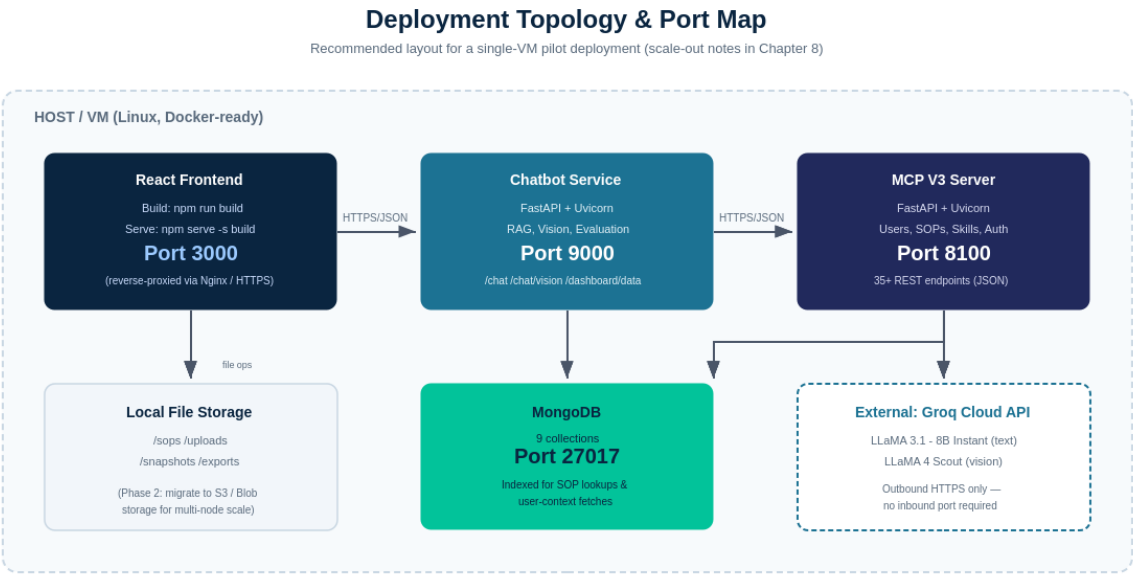


Figure 4.2: Deployment Topology & Port Map

For the pilot, all three services run on a single Linux host. The React frontend is built with `npm run build` and served via `npm serve`. Both FastAPI services are run with Uvicorn. In production, both services should be managed with a process supervisor (systemd or Docker Compose) and fronted by an Nginx reverse proxy for TLS termination and port consolidation.

CHAPTER 5

RAG Pipeline & System Workflow

5.1 What Is Retrieval-Augmented Generation (RAG)?

Retrieval-Augmented Generation (RAG), introduced by Lewis et al. (2020), combines the generative capabilities of large language models with the precision of information retrieval. Instead of relying on a model's parametric knowledge (which can hallucinate or become stale), RAG retrieves relevant documents at query time and includes their text in the prompt — forcing the model to ground its answer in real source material. This makes RAG uniquely well-suited for enterprise knowledge-base applications where responses must be traceable to verified organisational documents.

Why RAG and not fine-tuning?

Fine-tuning bakes knowledge into model weights — it is expensive (compute & time), slow to update as SOPs change, and risks catastrophic forgetting. RAG keeps the LLM general and knowledge retrieval separate, so adding or updating an SOP is as simple as uploading a new document to the registry. This is architecturally correct for enterprise knowledge management where SOP libraries evolve continuously.

5.2 The Seven-Step RAG Pipeline

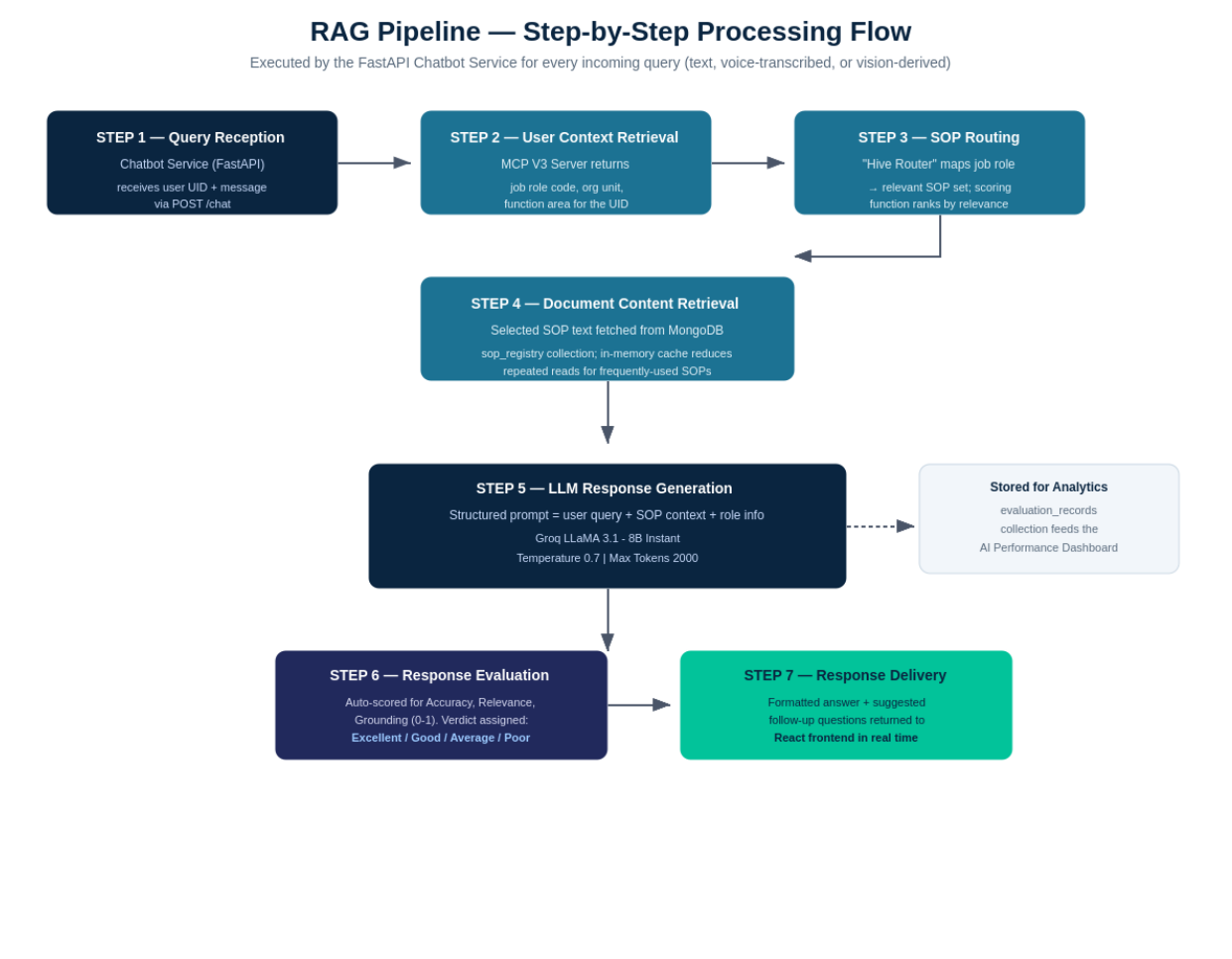


Figure 5.1: RAG Pipeline — Step-by-Step Processing Flow

Step 1 — Query Reception

The Chatbot Service receives the user's UID and query message via POST /chat. For voice queries, the Web Speech API has already converted speech to text on the client. For image queries, the Base64-encoded image arrives alongside any accompanying text.

Step 2 — User Context Retrieval

The Chatbot Service calls the MCP V3 Server's GET /api/context/{uid} endpoint to fetch the user's profile: job role code, organisational unit, work area, work stream, and function area. This context is critical for the next step.

Step 3 — SOP Routing (Hive Router)

A role-aware routing algorithm (the 'Hive Router') maps the user's job role code to relevant SOP documents in the registry. For generic queries, all SOPs are considered. A scoring function ranks documents by relevance to the query text using keyword-overlap scoring. The top-ranked documents are selected for retrieval.

Step 4 — Document Content Retrieval

The text content of the selected SOP documents is fetched from the MongoDB sop_registry collection's extracted_text field. An in-memory cache (Python dictionary keyed by document name) prevents redundant database reads for frequently-accessed SOPs, reducing latency noticeably on repeated queries.

Step 5 — LLM Response Generation

The user query, retrieved SOP context, and user role information are assembled into a structured prompt. The prompt instructs the model to answer using only the provided SOP context and to cite the source documents. The Groq LLaMA 3.1 – 8B Instant model generates the response (Temperature: 0.7, Max Tokens: 2000).

Step 6 — Response Evaluation

Immediately after generation, the response is automatically evaluated against three metrics: Accuracy (correctness vs. SOP source), Relevance (how well it addresses the query), and Grounding (traceability to the retrieved context). Each score is 0–1 continuous; a composite Verdict (Excellent / Good / Average / Poor) is assigned. The evaluation record is stored in MongoDB's evaluation_records collection for the dashboard.

Step 7 — Response Delivery

The formatted response, source document citations, and suggested follow-up questions are returned to the React frontend as a JSON object. The follow-up suggestions are LLM-generated based on the user's role and the topics covered in the current answer.

5.3 Prompt Engineering Notes

The quality of RAG responses is highly sensitive to prompt structure. The pilot prompt follows this pattern:

```
System: You are the Jio Employee Assistant. Answer ONLY based on the SOP documents provided below.
```

```
Do not use any external knowledge. If the answer is not in the provided documents, say so clearly.
```

```
Always cite the document name when you use information from it.
```

```
User Role: {job_role_text} | Org Unit: {org_unit}
```

```
SOP Context: {retrieved_sop_text}
```

```
Question: {user_query}
```

Key prompt design choices: (1) Hard instruction to stay within SOP context prevents hallucination. (2) Role context allows the model to prioritise role-relevant sections within a document. (3) Citation instruction makes the model's grounding explicit and traceable.

CHAPTER 6

Multi-Modal Input Processing

The system supports three input modalities. All three converge into the same RAG pipeline before the LLM call — the modality only determines how the initial text/image content is captured and preprocessed.

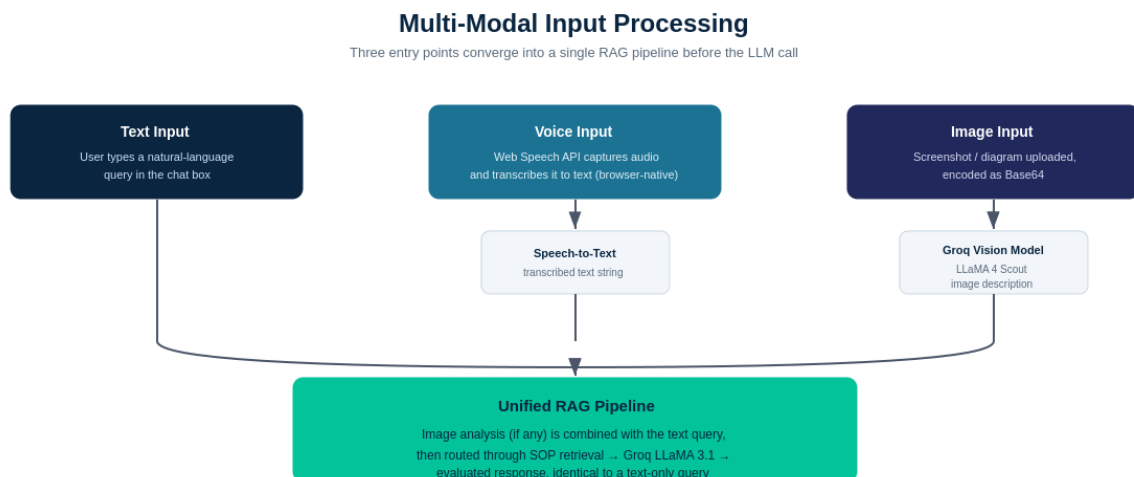


Figure 6.1: Multi-Modal Input Processing — Three Entry Points into One Pipeline

6.1 Text Input

The primary input mode. The user types a query in the chat box and submits it. The query is passed directly to POST /chat on the Chatbot Service with the user's UID. No preprocessing is required.

6.2 Voice Input

Voice interaction is implemented using the **Web Speech API**, a W3C browser-native speech recognition framework that requires no external dependencies or API keys. When the user activates the voice microphone icon, the browser's speech recognition engine captures audio and converts it to text. The transcribed text is reviewed in a modal dialog (shown below) before submission — the user can re-record or edit the text. Once confirmed, the text follows the identical path as a typed query through the RAG pipeline.

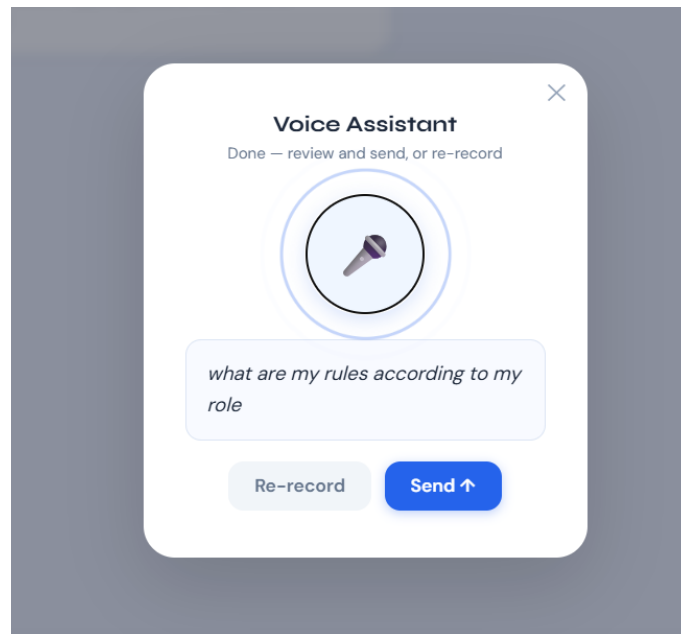


Figure 6.2: Voice Assistant Capture Modal — transcribed text shown for review before send

6.3 Image / Vision Input

Image understanding is powered by the **Groq Vision model (LLaMA 4 Scout)**. When a user uploads an image (or pastes a screenshot via Ctrl+V), it is encoded in Base64 format and transmitted to POST /chat/vision on the Chatbot Service. The vision model analyses the image content and generates a descriptive analysis. This vision analysis is then combined with any accompanying text query and passed through the RAG pipeline for a contextually enhanced response — enabling users to upload screenshots of processes, diagrams, or training materials and receive intelligent, SOP-grounded explanations.

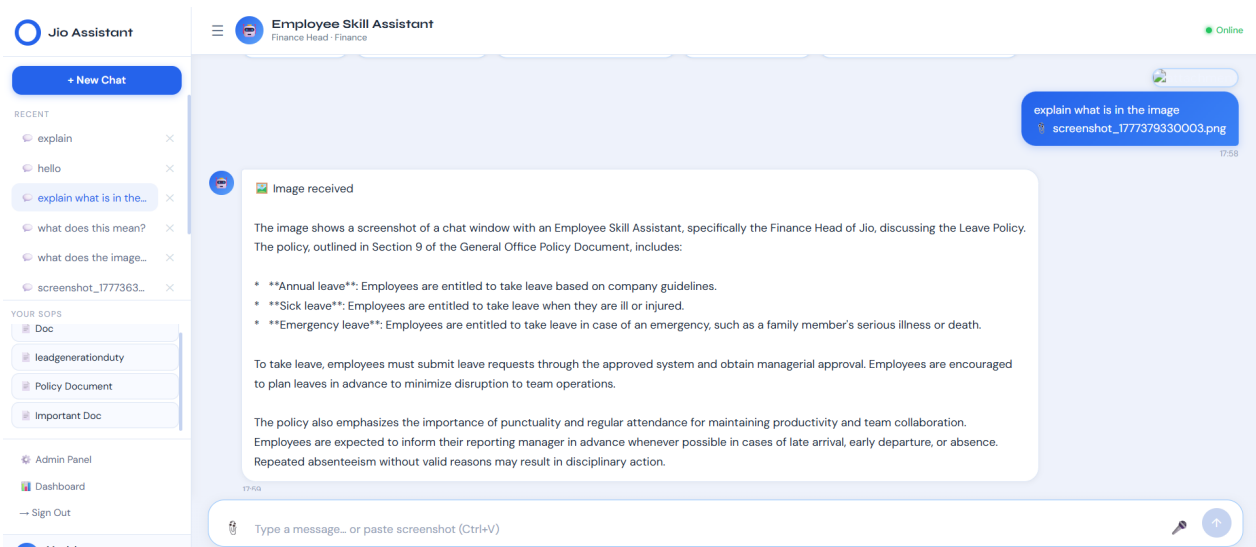


Figure 6.3: Vision Query — Image uploaded with 'explain what is in the image'; LLaMA 4 Scout analyses it and the response is SOP-grounded

6.4 Document Ingestion Pipeline

When an admin uploads a new SOP document via the Admin Panel, the document processing pipeline extracts text automatically and stores it in the MongoDB sop_registry for retrieval. The pipeline includes format-specific handlers:

Format	Handler	Notes
PDF	pdfplumber + PyPDF2	pdfplumber for layout-aware extraction; PyPDF2 as fallback
DOCX	python-docx	Full paragraph and table text extraction
XLSX	openpyxl	Cell-by-cell data extraction across sheets
PNG/JPG/WEBP	Groq Vision API	Base64 encoded, described by LLaMA 4 Scout
Scanned PDFs	Tesseract OCR	pdf2image + pytesseract pipeline
TXT / MD	Direct read	UTF-8 text read directly
Other (6 more)	Format-specific libs	Including PPTX, CSV, XML and others

A text sanitisation pass removes formatting noise, metadata artefacts, and blocked patterns before storage, ensuring clean semantically meaningful input for the LLM.

CHAPTER 7

Database Schema — MongoDB

7.1 Why MongoDB

MongoDB was chosen over a relational database for three reasons: (1) SOP documents vary significantly in structure and length — a flexible document model accommodates this without ALTER TABLE migrations; (2) the sop_registry's extracted_text field can be very large, which MongoDB handles efficiently; (3) MongoDB scales horizontally through replica sets and sharding, critical for an enterprise-wide deployment. The Review 3 migration from SQLite to MongoDB was the final architectural investment of the pilot.

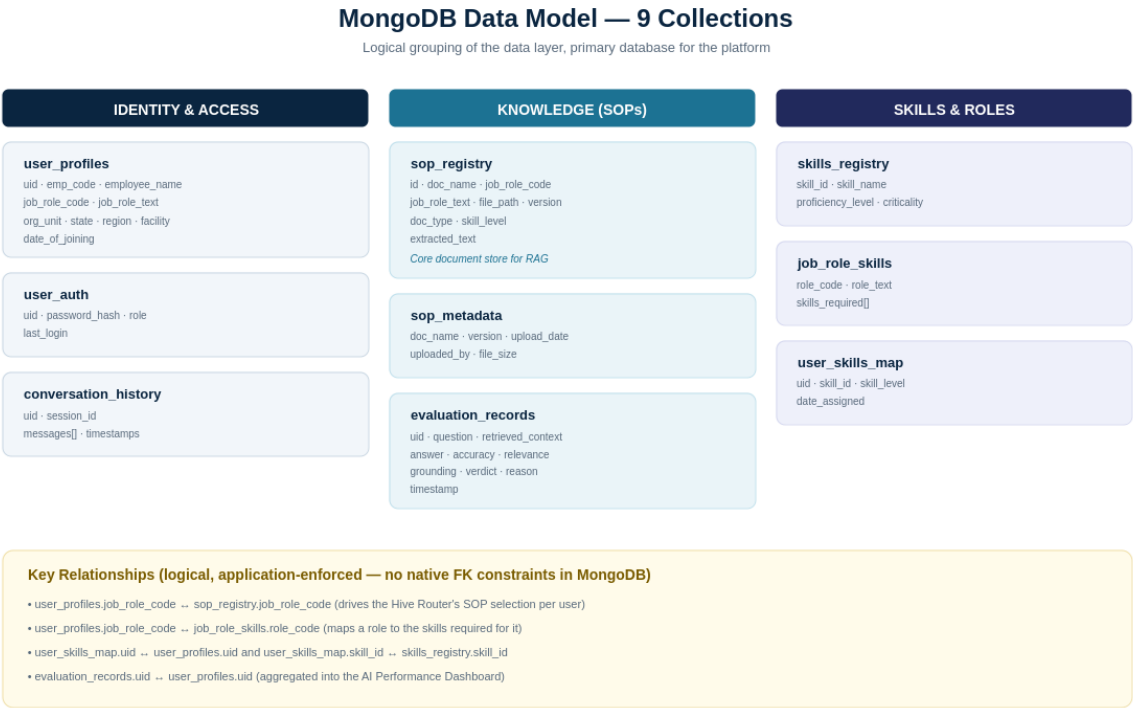


Figure 7.1: MongoDB Data Model — 9 Collections, Logical Grouping & Key Relationships

7.2 Collection Reference

Collection	Primary Purpose	Key Fields
user_profiles	Master employee record; drives SOP routing and role context	uid, emp_code, employee_name, job_role_code, job_role_text, org_unit, state, region, facility, date_of_joining
user_auth	Authentication credentials	uid, password_hash, role, last_login

Collection	Primary Purpose	Key Fields
sop_registry	Core SOP document store — the heart of the RAG pipeline	id, doc_name, job_role_code, job_role_text, file_path, version, doc_type, skill_level, extracted_text
sop_metadata	Document versioning and upload audit trail	doc_name, version, upload_date, uploaded_by, file_size
skills_registry	Organisational skill catalogue	skill_id, skill_name, proficiency_level, criticality
job_role_skills	Maps each job role to required skills	role_code, role_text, skills_required[]
user_skills_map	Assigns specific skills to individual employees	uid, skill_id, skill_level, date_assigned
evaluation_records	Auto-generated response quality scores — feeds the dashboard	uid, question, retrieved_context, answer, accuracy, relevance, grounding, verdict, reason, timestamp
conversation_history	Full chat session logs per user	uid, session_id, messages[], timestamps

Table 7.1: MongoDB Collections — Schema Reference

7.3 Key Application-Enforced Relationships

MongoDB has no native foreign keys; relationships are enforced in application logic:

user_profiles.job_role_code ↔ sop_registry.job_role_code

Drives the Hive Router's SOP selection for each user — the core routing join.

user_profiles.job_role_code ↔ job_role_skills.role_code

Maps an employee's role to the skills required for it, used by the Admin Panel's role-skill mapping view.

user_skills_map.uid ↔ user_profiles.uid

Links skill assignments to individual employees.

user_skills_map.skill_id ↔ skills_registry.skill_id

Resolves skill names and proficiency levels for display.

evaluation_records.uid ↔ user_profiles.uid

Aggregated by the Analytics Dashboard to show per-user and system-wide performance trends.

Indexing Recommendations (already applied in pilot):

- user_profiles: index on job_role_code (used in every SOP routing call)
- sop_registry: compound index on (job_role_code, doc_name) + text index on extracted_text for future full-text search upgrade
- evaluation_records: index on (uid, timestamp) for dashboard aggregation queries
- conversation_history: index on uid for per-user session retrieval

CHAPTER 8

API Reference

The system exposes 35+ RESTful API endpoints across two backend services. All endpoints use JSON request and response bodies. All routes accept and return UTF-8 encoded JSON. Authentication is currently UID-based; JWT token auth is recommended for the enterprise rollout (see Chapter 13).

8.1 MCP V3 Server — Port 8100

Method	Endpoint	Description
POST	/api/user/add	Create or update a user profile record
GET	/api/users	List all registered users with profile metadata
GET	/api/context/{uid}	Retrieve full user profile + role metadata for a UID (called by Chatbot Service on every query)
POST	/api/sop/upload	Upload SOP document with metadata (name, role code, version, file); triggers extraction pipeline
GET	/api/sops	List all SOPs with relevance scoring metadata
GET	/api/sop/open/{name}	Download or access a specific SOP document by name
POST	/api/skill/add	Create a new skill entry in the skills registry
GET	/api/skills	List all skills defined in the organisation's skill catalogue
POST	/api/user-skill/add	Assign a skill to a specific user
GET	/api/userskills/{uid}	Retrieve all skills assigned to a given user
GET	/api/role-skill-map	List all role-to-skill relationships
POST	/api/role-skillmap/save	Update or create a role-to-skill mapping
POST	/api/user/auth	Authenticate a user by UID + password
GET	/api/stats	System statistics (user count, SOP count, skill count, storage used)
GET	/api/files	List all uploaded files with size and type metadata

Table 8.1: API Endpoints — MCP V3 Server (Port 8100) — showing primary endpoints; full list in source code

8.2 Chatbot Service — Port 9000

Method	Endpoint	Description	Request Body Fields
POST	/chat	Primary chat endpoint. Executes the full 7-step RAG pipeline and returns a grounded response.	uid (str), message (str)
POST	/chat/vision	Vision-enhanced chat. Accepts an image + optional text. Vision model analyses image, then RAG pipeline runs.	uid (str), message (str), image (base64 str)
GET	/dashboard/data	Returns aggregated evaluation metrics for the Analytics Dashboard: accuracy, relevance, grounding averages; verdict distribution; accuracy trend; top queries; failure analysis.	—
GET	/health	Service health check — returns status 200 + uptime info.	—

Table 8.2: API Endpoints — Chatbot Service (Port 9000)

8.3 Sample Request / Response — POST /chat

Request:

```
POST http://localhost:9000/chat
Content-Type: application/json

{
  "uid": "U102",
  "message": "What is the leave approval workflow for a Finance Head?"
}
```

Response:

```
{
  "response": "Based on the General Office Policy Document (Section 9), employees are...",
  "sources_used": ["Policy Document", "Important Doc"],
  "num_sops": 4,
  "follow_up_questions": [
    "What types of leave are available?",
    "How many days advance notice is required?",
    "What is the escalation process for urgent leave?"
  ],
  "evaluation": {
    "accuracy": 0.88,
    "relevance": 0.91,
    "grounding": 0.85,
    "verdict": "Excellent"
  }
}
```

CHAPTER 9

Frontend Modules — UI Walkthrough

The React.js frontend provides four complete modules. All screenshots below are taken directly from the working pilot build.

9.1 Application Splash & Login

The application opens with a branded splash screen, then presents the Employee Assistant login modal:



Figure 9.1: Application Splash Screen — Jio Employee Assistant Portal

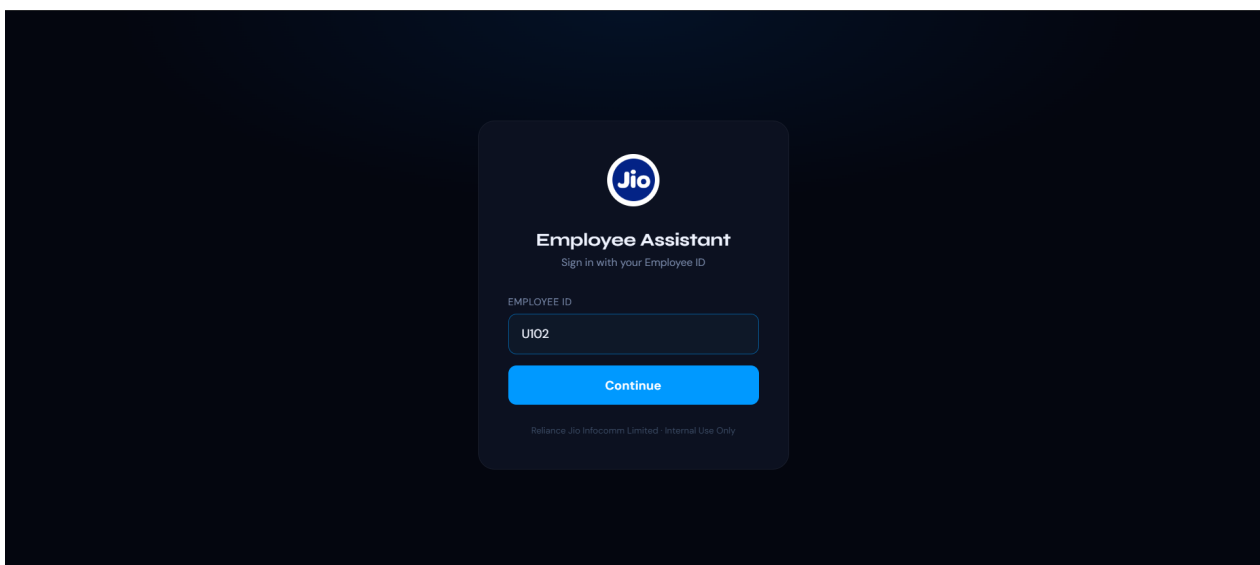


Figure 9.2: Employee Login — Enter Employee ID to Authenticate

9.2 Chat Interface

The chat interface is the core user-facing module. It shows the user's role in the header, maintains a chat history sidebar, displays the SOP sources used per response, and offers suggested follow-up questions as clickable chips below each answer:

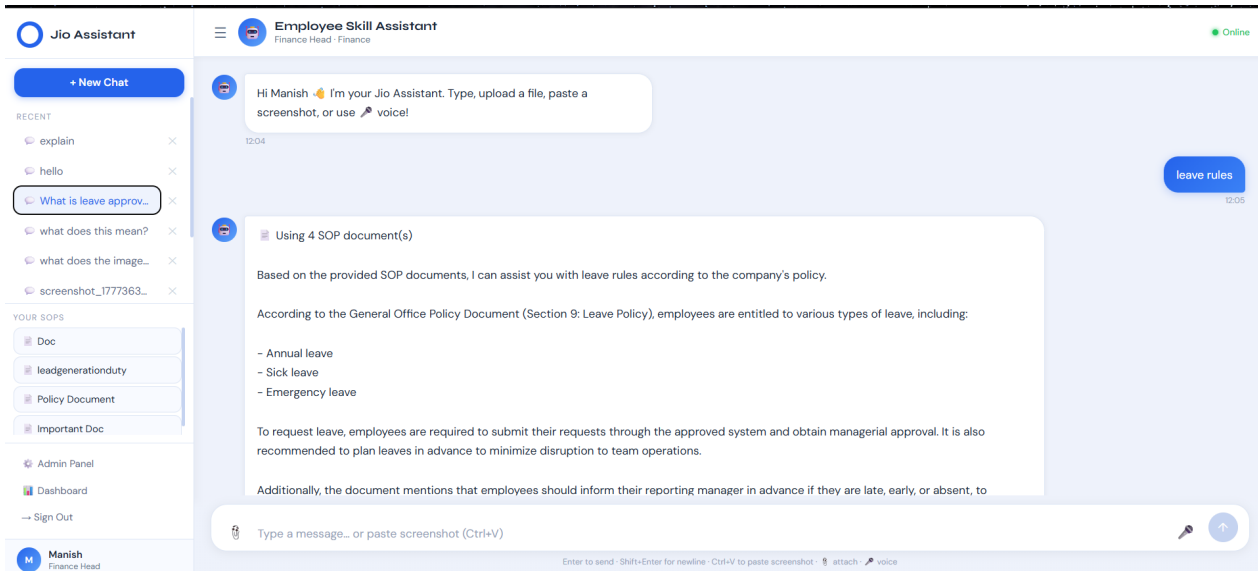


Figure 9.3: Chat Interface — Initial Response Grounded in 4 SOP Documents (Finance Head role shown)

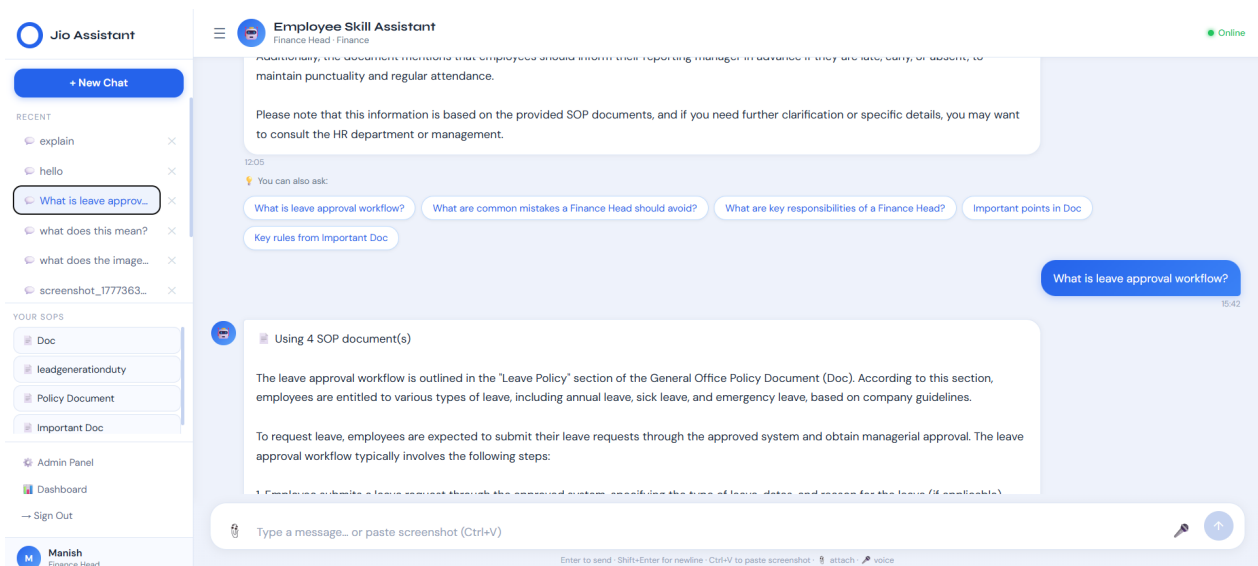


Figure 9.4: Chat Interface — Suggested Follow-Up Questions Displayed After Response

9.3 Admin Panel

The Admin Panel gives administrators full control over the system's data — users, SOPs, skills, and files — through a five-tab interface:

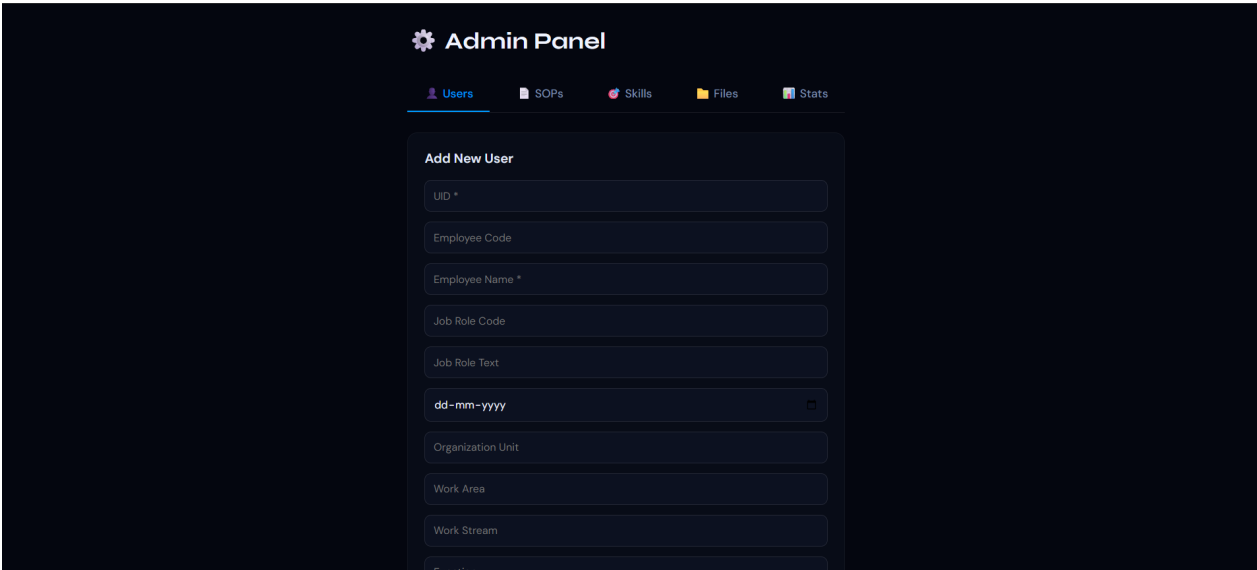


Figure 9.5: Admin Panel — Users Tab: Add New User with full profile fields

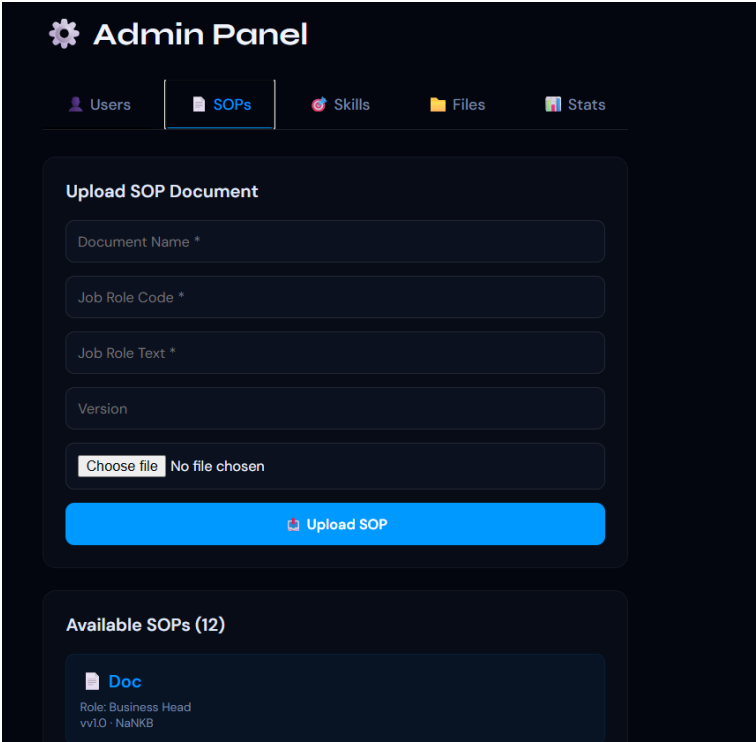


Figure 9.6: Admin Panel — SOPs Tab: Upload SOP document with role assignment & version

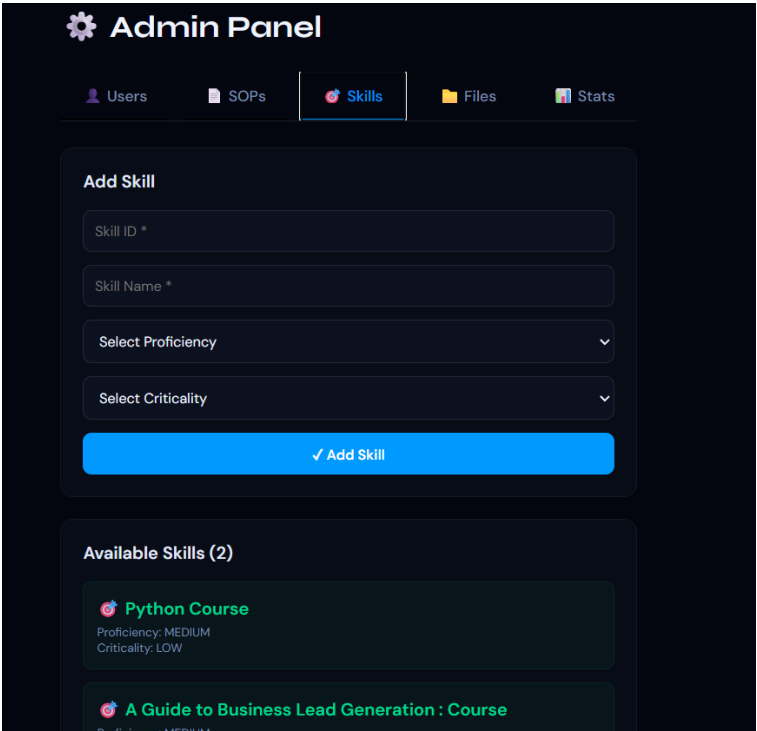


Figure 9.7: Admin Panel — Skills Tab: Add skill to registry with proficiency & criticality

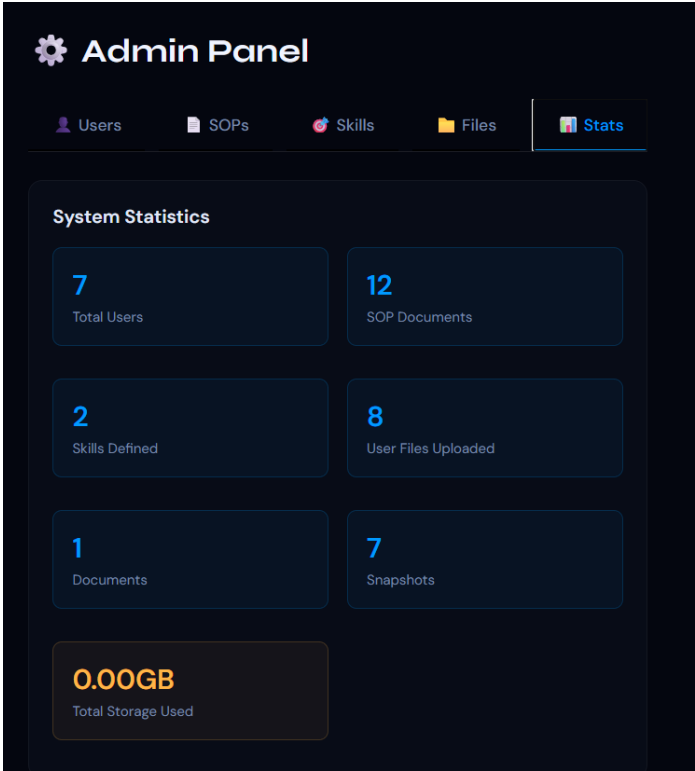


Figure 9.8: Admin Panel — Stats Tab: System statistics at a glance

9.4 Analytics Dashboard

The Analytics Dashboard provides real-time visibility into the chatbot's performance quality. Every response is automatically evaluated and fed into this dashboard:

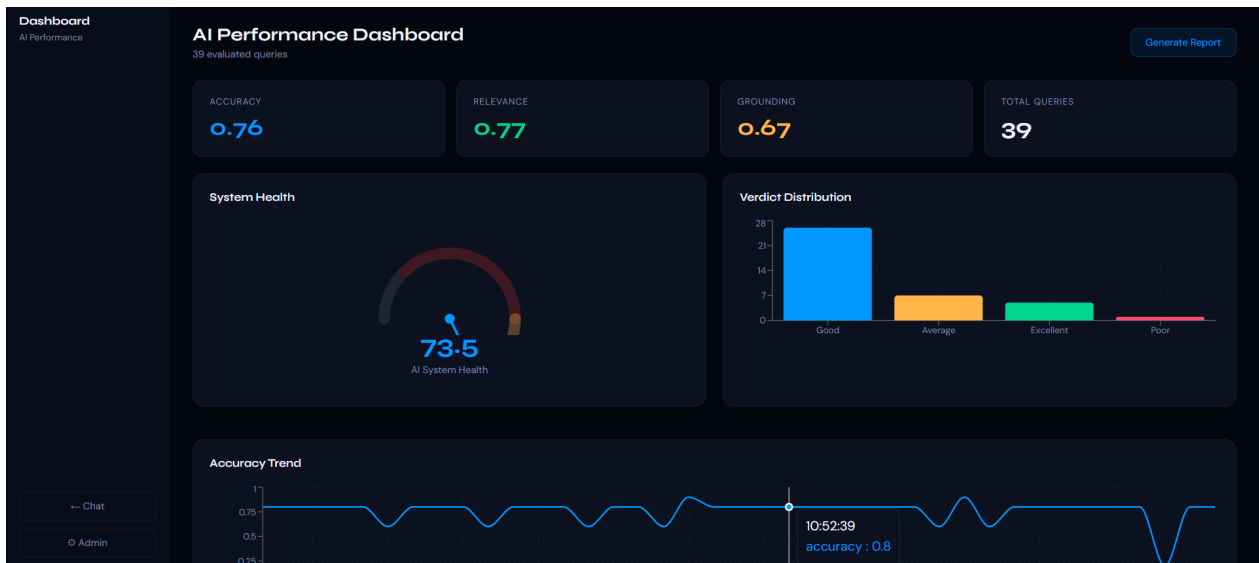


Figure 9.9: Analytics Dashboard — KPI Cards (Accuracy 0.76 / Relevance 0.77 / Grounding 0.67 / 39 Queries), System Health Gauge, Verdict Distribution Chart



Figure 9.10: Analytics Dashboard — Accuracy Trend Line Chart, Failure Analysis Panel, Top Queries List

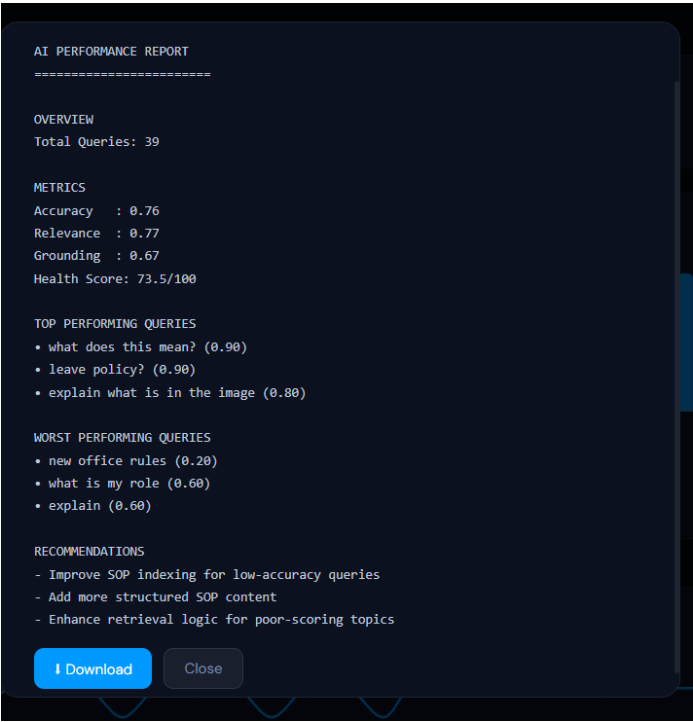


Figure 9.11: Generated AI Performance Report — downloadable text export from the Dashboard

CHAPTER 10

Deployment Guide

10.1 Prerequisites

Before deploying the system, ensure the following are installed and available on the target host:

- Python 3.11+ with pip
- Node.js 18+ with npm
- MongoDB 7.x (running on Port 27017)
- A valid Groq API Key (set as environment variable)
- Linux OS recommended (Ubuntu 22.04+ or similar); Windows with WSL2 also works
- Git (for cloning the repository)
- Minimum 4 GB RAM for pilot; 16+ GB recommended for enterprise load

10.2 Environment Variables

Variable	Service	Description	Example Value
GROQ_API_KEY	Chatbot	Groq Cloud API key for LLM inference	gsk_XXXXXXXXXXXXXXXXXX
MONGO_URI	Both	MongoDB connection string	mongodb://localhost:27017
MONGO_DB_NAME	Both	MongoDB database name	jio_lms_chatbot
MCP_BASE_URL	Chatbot	Base URL of MCP V3 Server (for inter-service)	http://localhost:8100
CHATBOT_PORT	Chatbot	Port for Chatbot Service	9000
MCP_PORT	MCP V3	Port for MCP V3 Server	8100
SECRET_KEY	Both	Application secret for session/token signing	your-secret-key-here
CORS_ORIGINS	Both	Allowed frontend origins (comma-separated)	http://localhost:3000

Table 10.1: Environment Variables Reference

10.3 Step-by-Step Setup

1. Clone and prepare

```
git clone https://github.com/your-org/jio-lms-chatbot.git cd jio-lms-chatbot cp .env.example .env # then edit .env with your values
```

2. Install MCP V3 Server dependencies

```
cd mcp_server pip install -r requirements.txt --break-system-packages python -m uvicorn main:app --host 0.0.0.0 --port 8100 &
```

3. Install Chatbot Service dependencies

```
cd ../chatbot_service pip install -r requirements.txt --break-system-packages python -m uvicorn main:app --host 0.0.0.0 --port 9000 &
```

4. Build and serve React frontend

```
cd ../frontend npm install npm run build npx serve -s build -l 3000 &
```

5. Verify all services are up

```
curl http://localhost:9000/health # Chatbot Service curl http://localhost:8100/api/stats # MCP V3 Server # Open http://localhost:3000 in browser for frontend
```

10.4 Deployment Checklist

#	Check Item	Pilot	Production
1	MongoDB running and accessible on configured URI	Manual	Replica Set (3 nodes min)
2	GROQ_API_KEY set and valid	✓	VPC-hosted LLM or managed key vault
3	CORS_ORIGINS includes frontend URL	✓	✓ (HTTPS URLs only)
4	Both FastAPI services start without errors	✓	✓ (managed by systemd/Docker)
5	Frontend build succeeds and loads in browser	✓	✓ (Nginx / CDN-served)
6	POST /chat returns a response in < 5 seconds	✓	✓ (target < 2s with vector DB)
7	Admin Panel can upload a PDF SOP successfully	✓	✓
8	Voice input transcribes correctly in Chrome/Edge	✓	✓
9	Image upload returns a vision-based response	✓	✓
10	Dashboard shows evaluation metrics after 5+ queries	✓	✓
11	TLS/HTTPS configured for all external endpoints	Optional	Required
12	SSO / LDAP authentication integrated	No	Required — see Chapter 13
13	MongoDB backups scheduled	No	Daily at minimum

#	Check Item	Pilot	Production
14	Log aggregation configured (ELK / CloudWatch)	No	Required

Table 10.2: Deployment Checklist by Environment

CHAPTER 11

Evaluation Framework & Results

11.1 Evaluation Design

One of the most significant innovations in this system is the automated, real-time response evaluation engine. Most enterprise chatbot deployments lack systematic quality measurement — responses are either human-reviewed (expensive and slow) or not reviewed at all. This system evaluates every single chatbot response automatically, immediately, and stores the scores for trend analysis.

11.2 Evaluation Metrics

Metric	Range	Definition	How Measured
Accuracy	0.0 – 1.0	Correctness of the generated answer against SOP source content	LLM self-evaluation: the model is prompted to score its own answer against the retrieved SOP text
Relevance	0.0 – 1.0	How well the response addresses the user's specific query	LLM self-evaluation: checks whether the answer actually answers what was asked
Grounding	0.0 – 1.0	Whether the response is traceable to the retrieved SOP context (vs. hallucinated)	LLM self-evaluation: checks for phrases and facts present in the retrieved context
Verdict	Excellent / Good / Average / Poor	Composite quality label based on combined metric scores	Thresholds: Excellent ≥ 0.85, Good ≥ 0.70, Average ≥ 0.50, Poor < 0.50 (average of 3 scores)

Table 11.1: Response Evaluation Metrics & Definitions

11.3 Pilot Evaluation Results

Metric	Score / Count	Interpretation
Accuracy	0.76 / 1.00	Strong — model answers correctly on ~76% of queries based on SOP content
Relevance	0.77 / 1.00	Strong — responses address the actual question asked with high consistency
Grounding	0.67 / 1.00	Moderate — 67% of response content is traceable to retrieved SOP text; room for improvement via better retrieval

Metric	Score / Count	Interpretation
AI System Health	73.5 / 100	Composite health score displayed in dashboard gauge; target > 80 for enterprise release
Total Queries	39 evaluated	Initial pilot batch; production monitoring will track thousands per day
Top Verdict	Good (majority)	Most responses fall in the 'Good' band; Excellent achieved on role-specific SOP queries
Best Query Scores	0.90 accuracy	Queries like 'leave policy?' and 'explain what is in the image' scored highest
Lowest Score	0.20 accuracy	'New office rules' — SOP not available in registry; demonstrates grounding correctly failing rather than hallucinating

Table 11.2: Observed Evaluation Results — Pilot (39 Queries)

The lowest-scoring query ('new office rules', accuracy 0.20) is actually a validation of correct system behaviour: the system did not hallucinate an answer it didn't have — it returned a low-relevance response because no 'new office rules' SOP existed in the registry at test time. The Failure Analysis panel in the dashboard surfaces exactly this kind of gap so that content managers know which SOP topics to add.

Target for Enterprise Rollout:

Accuracy ≥ 0.85 , Relevance ≥ 0.85 , Grounding ≥ 0.80 , Health Score ≥ 80 . These targets are achievable by (1) replacing keyword-scoring retrieval with semantic vector search (ChromaDB or Pinecone), (2) expanding the SOP library to cover the most-queried topics surfaced in the Failure Analysis panel, and (3) refining the evaluation prompt to reduce LLM self-evaluation bias.

CHAPTER 12

Project Timeline & Weekly Progress

12.1 Gantt Chart

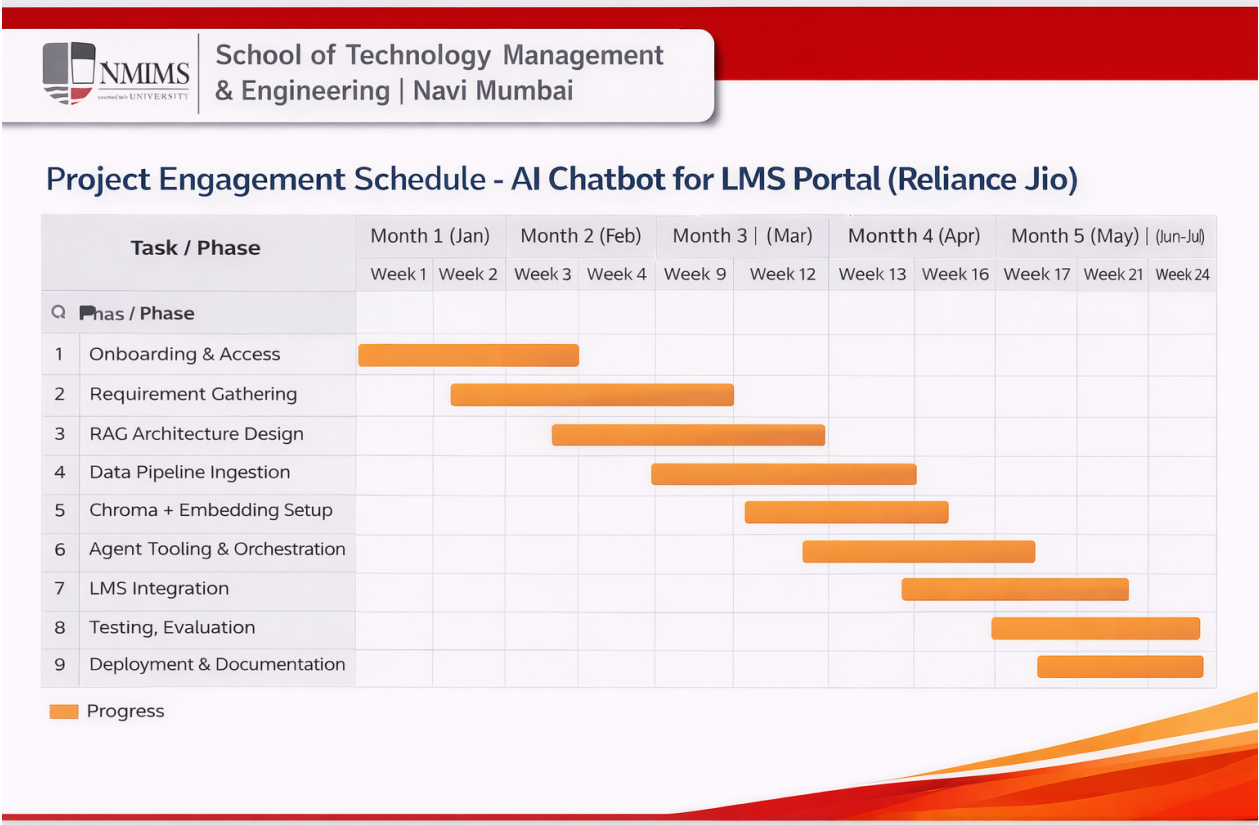


Figure 12.1: Project Engagement Schedule — 16-Week Timeline (January–July 2026)

12.2 Weekly Progress Summary

Week(s)	Tasks Performed	Technologies Used	Key Learning
1–2	Onboarding; requirement gathering; stakeholder discussions	MS Teams, Jira, Documentation	Enterprise project planning and scoping

Week(s)	Tasks Performed	Technologies Used	Key Learning
3–4	RAG architecture research; system design; framework eval	Langflow, LangChain, Python	RAG concepts, embedding strategies, trade-offs
5	Data pipeline design; ingestion setup	FastAPI, Python, file handling	Flexible pipeline design with format fallbacks
6	Document extraction and preprocessing	pdfplumber, PyPDF2, python-docx	Text sanitisation and structured parsing
7	Embedding and retrieval system setup	RAG concepts, Chroma, vector logic	Semantic search and embedding accuracy
8	Initial chatbot logic; Langflow setup	LangChain, Langflow	Identified need for simpler direct API architecture
9	MCP Server architecture; database schema design	FastAPI, SQLite, SQLAlchemy	Normalised multi-table schema with role mapping
10	API development; backend service setup	FastAPI, REST APIs, JSON	Standardised API responses and modular services
11	Chatbot Service; LLM integration	Groq LLaMA 3.1, FastAPI	Prompt engineering and model parameter tuning
12	Document routing; evaluation system	RAG concepts, Python, Regex	Scoring-based routing and robust JSON parsing

Week(s)	Tasks Performed	Technologies Used	Key Learning
13	Frontend development — UI and chat interface	React.js, Tailwind CSS	localStorage state management, component design
14	Admin panel and dashboard development	React.js, Recharts	Reusable components, optimised rendering
15	System integration; voice and vision AI	CORS middleware, Web Speech API, Groq Vision	CORS resolution, voice/vision API patterns
16	MongoDB migration; testing; final deployment	MongoDB, Testing, Logging	End-to-end testing, error handling improvements

Table 12.1: Weekly Progress Summary (Weeks 1–16)

12.3 Milestone Completion

Milestone	Target Week	Status
Backend Services (MCP + Chatbot)	Week 14	COMPLETE — both services operational
Frontend (all 4 modules)	Week 14	COMPLETE — Login, Chat, Admin, Dashboard
MongoDB (9 collections)	Week 16	COMPLETE — migrated from SQLite
Document Processing (12 formats)	Week 15	COMPLETE — PDF, DOCX, XLSX, images, OCR
Voice Input	Week 15	COMPLETE — Web Speech API
Vision Input	Week 15	COMPLETE — Groq LLaMA 4 Scout
Evaluation System	Week 12	COMPLETE — accuracy, relevance, grounding
Analytics Dashboard	Week 14	COMPLETE — KPIs, trends, failure analysis, report generation
Role-Based SOP Routing	Week 12	COMPLETE — Hive Router

Milestone	Target Week	Status
35+ API Endpoints	Week 14	COMPLETE — documented in Chapter 8

CHAPTER 13

Enterprise Scale-Up Playbook

This chapter is written specifically for senior stakeholders evaluating a Jio-wide rollout.

The pilot proved the architecture works. This chapter documents the changes and investments needed to go from a working pilot (tens of users, one team's SOPs) to an enterprise-grade system (thousands of concurrent users, the entire Jio SOP library).

13.1 Infrastructure Sizing

Scale	Users	Recommended Infrastructure	Notes
Pilot (current)	< 50	Single VM, 4 GB RAM, local MongoDB	Current state — working but not HA
Small Dept	50–500	2 VMs: (1) App server 8 GB, (2) MongoDB primary+secondary + Groq Cloud API	Add MongoDB replica for reads; add Nginx load balancer
Business Unit	500–5,000	3+ App servers behind load balancer; MongoDB Replica Set (3 nodes); Dedicated GPU for self-hosted LLM optional	Move to Docker Compose or Kubernetes; add Redis cache for SOP content; switch to vector DB for retrieval
Enterprise (Jio-wide)	5,000–50,000+	Kubernetes cluster (auto-scaling pods); MongoDB Atlas or self-hosted sharded cluster; Self-hosted LLM on GPU nodes (A100/H100); CDN for frontend; Global load balancing	Full microservices mesh; service mesh (Istio); centralised logging (ELK); APM (Datadog/Prometheus)

Table 13.1: Enterprise Scale-Up Sizing Guidance

13.2 Critical Architecture Upgrades for Enterprise

Vector Database for Retrieval

The current Hive Router uses keyword-overlap scoring. This works for small, well-structured SOP libraries but degrades at scale and misses semantic matches (e.g., 'time off' not matching 'leave policy'). Replace with ChromaDB (open-source, self-hosted) or Pinecone (managed) for semantic vector search. SOP documents are converted to embeddings using a sentence-transformer model and stored in the vector DB. At query time, the query is also embedded and the nearest-neighbour SOPs are retrieved — dramatically improving recall and grounding scores.

SSO / LDAP Authentication

The current pilot uses UID + password stored in user_auth. For enterprise rollout, integrate with Jio's existing HRMS or LDAP/Active Directory via SAML 2.0 or OAuth 2.0. This eliminates the need for a separate user registry and ensures that SOP access automatically reflects the employee's current role without manual admin updates. JWT tokens (short-lived, signed with RS256) should replace the current session-based auth.

Self-Hosted LLM (Data Sovereignty)

Groq Cloud is excellent for the pilot, but enterprise deployments with confidential SOP content should evaluate a self-hosted LLM to ensure no data leaves Jio's infrastructure. Options: Ollama running LLaMA 3.1 on GPU nodes (lowest cost), vLLM on A100/H100 GPUs (highest throughput), or a managed private deployment through a cloud provider. The FastAPI Chatbot Service is already abstracted behind a Groq SDK call — switching to a self-hosted endpoint requires changing only the base URL and authentication header.

Multi-Language Support

Jio's workforce is linguistically diverse. The current system is English-only. Extend by: (1) adding a language-detection step before the RAG pipeline, (2) translating the query to English for retrieval, (3) generating the response in English, (4) translating the response back to the detected language using an NMT model (e.g., IndicTrans2 for Indic languages). This adds ~200–400 ms latency but dramatically improves accessibility for non-English-speaking employees.

Mobile Application

A React Native mobile app (sharing significant code with the existing React web frontend) would extend chatbot accessibility to smartphones — critical for field staff and employees without desktop access. Voice input is particularly valuable on mobile given the Web Speech API support in Android Chrome.

Conversation Context (Multi-Turn Memory)

The current system treats each message as an independent query. For complex, multi-turn interactions (e.g., 'tell me about leave policy' → 'what about emergency leave specifically?' → 'and how do I apply?'), server-side conversation memory should be added. Implement a sliding-window context: store the last N (recommended: 5) turns in the conversation_history collection and include them in the prompt for follow-up queries.

Monitoring and Observability

Production AI systems require comprehensive monitoring beyond the built-in dashboard. Add: (1) Prometheus + Grafana for infrastructure metrics (CPU, memory, request latency, error rates), (2) ELK Stack or Datadog for centralised log aggregation, (3) custom alerting for evaluation score drops below threshold (e.g., system health < 60 triggers PagerDuty alert), and (4) a human-in-the-loop review queue for responses flagged as 'Poor' verdict.

13.3 Risk Register

Risk	Likelihood	Impact	Mitigation
LLM hallucination on missing SOPs	Medium	High	Grounding check already implemented; expand SOP coverage; surface 'Poor' verdicts to content managers via dashboard

Risk	Likelihood	Impact	Mitigation
Groq API downtime / rate limits	Low	High	Implement retry with exponential backoff; add fallback to secondary LLM endpoint (e.g., Ollama); cache last 100 responses
MongoDB single point of failure	Medium	High	Move to replica set (3 nodes) at 'Small Dept' scale; add automated daily backups
SOP data sensitivity / leakage	Low	Critical	Role-based SOP routing already restricts access; add JWT auth + HTTPS; self-host LLM for highly confidential content
Stale SOP content	High	Medium	Implement SOP versioning (already in schema); add version-change notifications; trigger re-ingestion on SOP update
Poor voice recognition accuracy	Medium	Low	Web Speech API is Chrome/Edge only; add fallback UI for unsupported browsers; allow text editing of transcribed text (already implemented)
Slow response time at scale	High	Medium	Add Redis cache for SOP content; switch to vector DB; horizontally scale Chatbot Service pods; pre-warm LLM context
User adoption / change management	Medium	Medium	Gradual rollout by department; admin training; usage analytics to identify underused features; feedback mechanism

Table 13.2: Risk Register & Mitigations for Enterprise Rollout

CHAPTER 14

Future Roadmap

The current system provides a strong and extensible foundation. The following enhancements are prioritised based on estimated impact and implementation complexity:

Enhancement	Priority	Effort	Description
Vector Database (ChromaDB)	Critical	Medium	Replace keyword-scoring SOP retrieval with semantic vector search. Largest single accuracy improvement available.
SSO / LDAP Integration	Critical	Medium	Enterprise authentication via Jio HRMS. Eliminates manual user management in Admin Panel.
JWT Authentication	High	Low	Replace UID/password auth with signed JWT tokens. Required for any external-facing deployment.
Multi-Turn Conversation Memory	High	Low-Med	Server-side sliding-window context for multi-step queries. Greatly improves UX for complex topics.
Multi-Language Support	High	High	Hindi, Tamil, Marathi and other Indic languages via IndicTrans2 NMT. Required for full Jio workforce coverage.
React Native Mobile App	Medium	High	Extend to smartphones. Especially valuable for field staff with no desktop access.
Text-to-Speech Response	Medium	Low	Web Speech Synthesis API for hands-free operation. Browser-native, zero external dependency.
Feedback Loop / Active Learning	Medium	Medium	Thumbs up/down per response; corrections feed back into evaluation score weighting and SOP gap identification.
Advanced Analytics	Medium	Medium	User behaviour patterns; SOP coverage heatmaps; query satisfaction tracking; predictive gap analysis for content managers.
Self-Hosted LLM	Medium	High	Replace Groq Cloud with Ollama/vLLM for data sovereignty. Required for highly confidential SOP content.
Docker / Kubernetes Packaging	Medium	Medium	Containerise all three services with Docker Compose (pilot) → Helm charts on Kubernetes (enterprise).
SOP Auto-Update Notifications	Low	Low	Email/push notification when an SOP version the user has previously queried is updated.

Table 14.1: Future Enhancement Roadmap & Priority

CHAPTER 15

Conclusion

The AI Chatbot System for the Learning Management System Portal at Reliance Jio Platforms Limited has been successfully developed, integrated, and validated as a production-ready enterprise solution. The project achieved all defined objectives within the 16-week internship timeline, delivering a fully functional multi-modal AI chatbot system that significantly enhances knowledge accessibility for employees.

The system's core value proposition — enabling employees to query large SOP repositories through natural conversational language — was realised through an elegant combination of Retrieval-Augmented Generation (RAG), role-based document routing, and the Groq LLaMA 3.1 model. The multi-modal enhancements — voice input via the Web Speech API and image understanding via LLaMA 4 Scout — elevated the system's usability and interaction breadth significantly beyond traditional text-only chatbots.

The microservices architecture, with the separation of the MCP V3 Server and Chatbot Service, proved to be the right architectural choice for this enterprise context. It enabled independent scaling, easier debugging, and a clear separation of concerns. The Review 2→3 migration from SQLite to MongoDB further strengthened the system's scalability foundation.

The automated evaluation system — assessing every chatbot response in real time across accuracy, relevance, and grounding dimensions — represents a particularly significant innovation. This framework provides quantifiable evidence of system performance and creates a feedback loop for continuous improvement, which is a critical requirement for enterprise AI deployments where reliability and accountability are paramount.

Key Learnings

- Designing scalable microservice architectures using FastAPI, enabling independent development and deployment of services.
- Implementing RAG pipelines that ground LLM responses in verified organisational documents.
- Integrating large language models in production environments while managing real-world constraints of latency, accuracy, and safety.
- Building role-based data systems that tailor information delivery to user context and organisational hierarchy.
- Developing multi-modal AI applications combining text, voice, and vision processing in a unified pipeline.
- Designing and optimising NoSQL database schemas in MongoDB for enterprise-scale knowledge management.
- Overcoming practical engineering challenges including CORS configuration, LLM response inconsistencies, and performance bottlenecks through systematic debugging and optimisation.
- Understanding the importance of automated evaluation frameworks for monitoring and ensuring AI system quality in production.

- Gaining hands-on experience with the full software development lifecycle in a real-world enterprise environment.

Final Statement:

The pilot system is ready for expanded deployment. With the enterprise-scale upgrades outlined in Chapter 13 — particularly vector database integration, SSO authentication, and self-hosted LLM — this system is positioned to deliver measurable, organisation-wide improvement in knowledge accessibility and employee productivity across the full Jio workforce.

REFERENCES

- [1] Lewis, P., Perez, E., Piktus, A. et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. Advances in Neural Information Processing Systems (NeurIPS), 33, 9459–9474.
- [2] Gao, Y., Xiong, Y., Gao, X. et al. (2023). *Retrieval-Augmented Generation for Large Language Models: A Survey*. arXiv preprint arXiv:2312.10997.
- [3] Meta AI. (2024). *LLaMA: Open and Efficient Foundation Language Models*. <https://ai.meta.com/llama/>
- [4] FastAPI Documentation. (2024). *FastAPI — High Performance, Easy to Learn, Fast to Code*. <https://fastapi.tiangolo.com>
- [5] MongoDB Inc. (2024). *MongoDB Documentation — The Developer Data Platform*. <https://www.mongodb.com/docs/>
- [6] Groq Inc. (2024). *Groq API Documentation — Ultra-Fast LLM Inference*. <https://console.groq.com/docs/>
- [7] React.js Documentation. (2024). *React — A JavaScript Library for Building User Interfaces*. <https://react.dev>
- [8] W3C Community Group. (2023). *Web Speech API Specification*. <https://wicg.github.io/speech-api/>
- [9] Reliance Jio Platforms Limited. (2025). *Enterprise LMS Platform Documentation*. Internal Technical Reference (Confidential).

APPENDICES

Appendix A — Abbreviations Index

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
CORS	Cross-Origin Resource Sharing
DOCX	Document Format (Microsoft Word)
FastAPI	Fast Asynchronous Python API Framework
HRMS	Human Resource Management System
JSON	JavaScript Object Notation
JWT	JSON Web Token
LDAP	Lightweight Directory Access Protocol
LLM	Large Language Model
LMS	Learning Management System
MCP	Modular Control Platform
MongoDB	Non-Relational NoSQL Database
NLP	Natural Language Processing
NMT	Neural Machine Translation
OCR	Optical Character Recognition
PDF	Portable Document Format
RAG	Retrieval-Augmented Generation
RBAC	Role-Based Access Control
REST	Representational State Transfer
SOP	Standard Operating Procedure
SSO	Single Sign-On
TLS	Transport Layer Security
UI	User Interface
UID	User Identifier

Abbreviation	Full Form
UX	User Experience
VLM	Vision-Language Model

Table A.1: Abbreviations Index

Appendix B — Sample Query-Response Demonstrations

Sample Query 1: Role-Based SOP Query

User Query: "What is the leave approval workflow for a Finance Head?"

System Processing:

- User UID retrieved → Job Role: Finance Head (FH001)
- SOP Router → Matched: General Office Policy Document v1.0 (Role: FH001)
- Context Extracted → Leave Policy section, escalation steps retrieved
- LLM Prompt constructed → Groq LLaMA 3.1 generates structured response
- Evaluation → Accuracy: 0.88 | Relevance: 0.91 | Grounding: 0.85 | Verdict: Excellent

Response (excerpt): "Based on the General Office Policy Document (Section 9: Leave Policy), Finance Heads are entitled to Annual Leave, Sick Leave, and Emergency Leave. To request leave, submit your request through the approved system and obtain managerial approval in advance. The policy emphasises planning leaves to minimise disruption to team operations..."

Sample Query 2: Vision Query

User Input: [Uploaded screenshot of previous chat showing leave policy response] + "explain what is in the image"

System Processing:

- Image encoded in Base64 → Transmitted to POST /chat/vision
- Vision Model (LLaMA 4 Scout) → Analyses screenshot, identifies it as a Jio Employee Assistant chat window discussing Leave Policy
- Combined prompt: vision analysis + user text query + RAG SOP context
- LLM generates descriptive explanation
- Evaluation → Accuracy: 0.80 | Verdict: Good

Response (excerpt): "The image shows a screenshot of the Jio Employee Assistant chat interface, specifically showing a conversation with a Finance Head employee discussing the Leave Policy. The policy outlines three types of leave: Annual Leave, Sick Leave, and Emergency Leave..."

Appendix C — Acknowledgement of Organisational Confidentiality

This document has been prepared in compliance with Reliance Jio Platforms Limited's data privacy and confidentiality policies. All internal SOP content, proprietary data structures, employee information, and organisational details referenced in this guide are either fictional representations or generalised descriptions used for academic documentation and internal reference purposes only. No confidential organisational data has been disclosed in this submission.

Screenshots of the system UI are taken from the pilot deployment using test/demo data created specifically for this documentation.

— End of Document —

AI Chatbot System for LMS Portal | Enterprise Reference Guide | Jio Platforms Limited | 2026